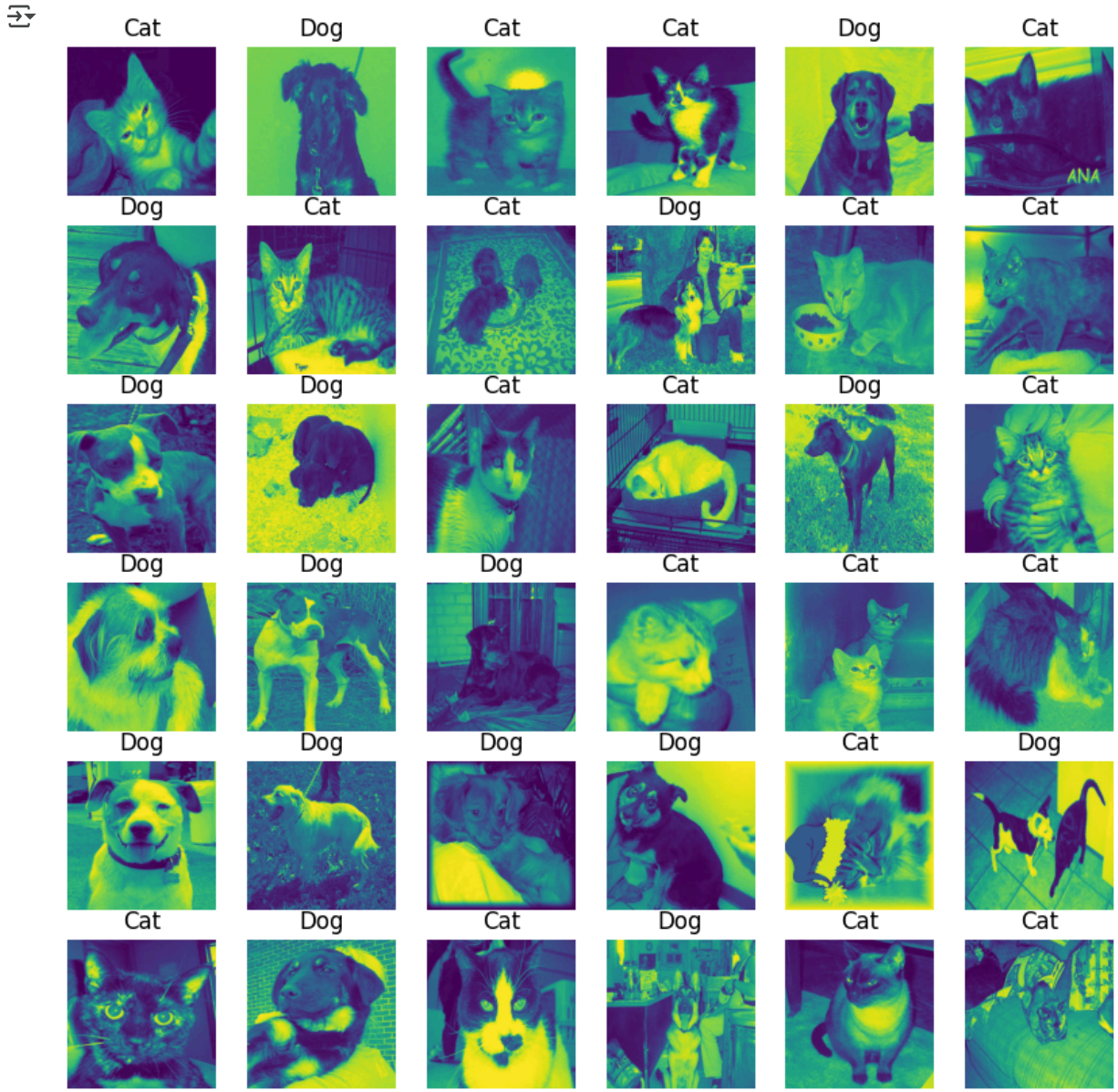
```
# Importing basic Library :
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os

# Sklearn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.datasets import make_regression
from sklearn import metrics
from sklearn import decomposition
from sklearn import manifold
from tqdm.notebook import trange, tqdm

#TensorFlow and Keras Libraries
import tensorflow as tf
from tensorflow.keras.layers import Dense, Flatten, Conv2D
from tensorflow.keras import Model
import torch
from torchvision import transforms
from tensorflow.keras.utils import to_categorical
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.datasets as datasets
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
import os

# Hiding the warnings
import warnings
warnings.filterwarnings('ignore')

# Extra Libraries
import copy
import random
import time


# Setting up custom dataloader :
# Path to dataset
```

```
dataset_path = "/content/drive/MyDrive/Image_Data"

# Image transformations
transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.125])
])

class CatsDogsSnakesDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.images = []
        self.labels = []

        # Load all images from the "cats" folder
        cat_folder = os.path.join(root_dir, "cats")
        for file in os.listdir(cat_folder):
            if file.endswith(('jpg', 'png', 'jpeg')):
                self.images.append(os.path.join(cat_folder, file))
                self.labels.append(0)  # 0 for Cats

        # Load all images from the "dogs" folder
        dog_folder = os.path.join(root_dir, "dogs")
        for file in os.listdir(dog_folder):
            if file.endswith(('jpg', 'png', 'jpeg')):
                self.images.append(os.path.join(dog_folder, file))
                self.labels.append(1)  # 1 for Dogs

        # # Load all images from the "Snake" folder
        # snake_folder = os.path.join(root_dir, "snakes")
        # for file in os.listdir(snake_folder):
        #     if file.endswith(('jpg', 'png', 'jpeg')):
        #         self.images.append(os.path.join(snake_folder, file))
        #         self.labels.append(2)  # 2 for Snakes

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        # Load image and label
        img_path = self.images[idx]
        label = self.labels[idx]

        # Open image
        image = Image.open(img_path).convert("L")

        # Apply transformations if any
        if self.transform:
            image = self.transform(image)
```

```
        # Convert label to tensor
        label = torch.tensor(label, dtype=torch.long)

        # Return image and label
        return image, label

# Instantiate Dataset
dataset = CatsDogsSnakesDataset(dataset_path, transform=transform)

# DataLoader
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)

# Test the DataLoader
for images, labels in dataloader:
    print(f"Batch size: {images.shape}, Labels: {labels}")
    break
```

```
Batch size: torch.Size([64, 1, 256, 256]), Labels: tensor([0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0,
        1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1,
        1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1])
```

```
# Load one batch from DataLoader
images, labels = next(iter(dataloader))

# Define Class Names
class_names = ["Cat", "Dog", "Snake"]

# Plot the Images
plt.figure(figsize=(10, 10))

# Visualize 25 images (5x5 grid)
for i in range(36):
    plt.subplot(6, 6, i+1)
    img = images[i].permute(1, 2, 0)  # Change tensor shape from (C, H, W) -> (H, W, C)

    # Reverse normalization (optional)
    img = img * 0.125 + 0.5  # Because you normalized with mean=0.5 and std=0.125

    # Plot the image
    plt.imshow(img.numpy())
    plt.title(f"{class_names[labels[i].item()]}")
    plt.axis("off")

plt.show()
```




```
# Build a Simple CNN Model
class CNNClassifier(nn.Module):
    def __init__(self):
        super(CNNClassifier, self).__init__()

        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1)  # (B, 3, 256, 256) -> (B, 32, 256, 256)
```

```
            self.pool = nn.MaxPool2d(2, 2)                                      # (B, 32, 128, 128)
            self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1)  # (B, 64, 128, 128)
            self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1) # (B, 128, 64, 64)

            self.fc1 = nn.Linear(128 * 32 * 32, 256)  # Flatten
            self.fc2 = nn.Linear(256, 3)              # 3 classes: cat, dog, snake

        def forward(self, x):
            x = self.pool(F.relu(self.conv1(x)))
            x = self.pool(F.relu(self.conv2(x)))
            x = self.pool(F.relu(self.conv3(x)))

            x = x.view(x.size(0), -1)  # Flatten
            x = F.relu(self.fc1(x))
            x = self.fc2(x)

            return x


    # 2. Set Device, Model, Loss, Optimizer
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    model = CNNClassifier().to(device)

    criterion = nn.CrossEntropyLoss()  # Because 3 classes
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)


    device
```

```
device(type='cuda')
```

```
    # 3. Train the Model
    def train_model(model, dataloader, criterion, optimizer, device, epochs=5):
        model.train()
        for epoch in range(epochs):
            running_loss = 0.0
            correct = 0
            total = 0

            for images, labels in dataloader:
                images = images.to(device)
                labels = labels.to(device)

                optimizer.zero_grad()

                outputs = model(images)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()
```

```
            running_loss += loss.item()

            _, preds = torch.max(outputs, 1)
            total += labels.size(0)
            correct += (preds == labels).sum().item()

        epoch_loss = running_loss / len(dataloader)
        epoch_acc = 100 * correct / total
        print(f"Epoch {epoch+1}/{epochs} | Loss: {epoch_loss:.4f} | Accuracy: {epoch_acc:.2f}%")
    print("\nTraining Complete!")

# Train for 5 epochs
train_model(model, dataloader, criterion, optimizer, device, epochs=5)

# Save the trained model
torch.save(model.state_dict(), "cnn_classifier.pth")
```

```
Epoch 1/5 | Loss: 1.6848 | Accuracy: 52.30%
Epoch 2/5 | Loss: 0.6406 | Accuracy: 62.50%
Epoch 3/5 | Loss: 0.5602 | Accuracy: 70.90%
Epoch 4/5 | Loss: 0.4332 | Accuracy: 80.65%
Epoch 5/5 | Loss: 0.3361 | Accuracy: 85.10%

Training Complete!
```

```
# 4. Testing the Model
def test_model(model, dataloader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            _, preds = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (preds == labels).sum().item()

    accuracy = 100 * correct / total
    print(f"\n🎯 Test Accuracy: {accuracy:.2f}%")

# Testing (on same dataloader because no separate train/test split was mentioned)
test_model(model, dataloader, device)
```

```
🎯 Test Accuracy: 86.90%
```

```
# 5. Predict some samples
model.eval()

# Load one batch
images, labels = next(iter(dataloader))
images, labels = images.to(device), labels.to(device)

outputs = model(images)
_, preds = torch.max(outputs, 1)

plt.figure(figsize=(12, 12))
for idx in range(25):
    plt.subplot(5, 5, idx+1)
    img = images[idx].cpu().permute(1, 2, 0)
    img = img * 0.125 + 0.5  # De-normalize
    plt.imshow(img.squeeze(), cmap='gray')
    true_label = class_names[labels[idx].item()]
    pred_label = class_names[preds[idx].item()]
    plt.title(f"T: {true_label}\nP: {pred_label}", fontsize=8)
    plt.axis('off')

plt.tight_layout()
plt.show()
```


T: Cat
P: Cat
T: Cat
P: Cat
T: Dog
P: Dog
T: Cat
P: Dog
T: Dog
P: Dog
T: Cat
P: Cat
T: Dog
P: Dog
T: Cat
P: Cat
T: Cat
P: Cat
T: Cat
P: Cat
T: Dog
P: Dog
T: Dog
P: Dog
T: Cat
P: Dog
T: Cat
P: Cat
T: Dog
P: Dog
T: Dog
P: Dog
T: Dog
P: Dog
T: Dog
P: Dog
T: Dog
P: Dog
T: Dog
P: Dog